

Python & PDB Cheatsheet

如何开始调试

插入代码启动调试
<code>import pdb; pdb.set_trace()</code>
启动默认调试器pdb 不需要import pdb
<code>breakpoint()</code>
如不想修改代码 可修改运行指令
<code>python -m pdb script.py</code>

lineno是什么

lineno (行号) 指源代码文件中的行号 (从 1 开始) 在命令中使用时 它指定文件中的特定行
在当前文件的第 10 行设置断点
<code>b 10</code>
执行到当前函数的第 15 行
<code>until 15</code>

执行控制

执行当前行 进入被调用函数
<code>s(tep)</code>
执行到当前函数的下一行 不进入函数内部
<code>n(ext)</code>
继续运行直到断点
<code>c(ontinue)</code>
执行到当前函数返回
<code>r(eturn)</code>
执行到指定行号或当前行之后 若无参数, 执行完当前行 (含循环) 指定行号指的是当前函数中的行号
<code>unt(il) [lineno]</code>
保留断点重启程序 重启时不会重新编译、加载模块 不会检测代码变更
<code>run [args] / restart [args]</code>

断点管理

设置断点 支持条件 无参数则会列出所有当前断点的详细信息
<code>b(reak) [[[filename:]lineno function], condition]]</code>
myfile.py 的第 20 行设置断点
<code>b myfile.py:20</code>

mymod/utlis.py 的 add 函数入口设置断点
<code>b mymod.utlis.add</code>
在第 10 行设置断点 当 x > 5 时触发
<code>b 10 x > 5</code>
在 add 函数入口设置断点 当 x == 10 时触发
<code>b add x == 10</code>
在 myfile.py 第 10 行设置条件断点 (x > 5)
<code>b myfile.py:10 x > 5</code>
清除断点 无参数则会清除调试会话中设置的所有断点 不会修改或删除源代码中的 breakpoint()
<code>c(l(ear) [filename:lineno bnumber ...])</code>
禁用断点 (保留但不触发)
<code>disable bnumber ...</code>
启用断点
<code>enable bnumber ...</code>

检查和显示

显示堆栈 (箭头>表示当前帧)
<code>w(here) [count]</code>
在堆栈中上移动帧
<code>u(p) [count]</code>
在堆栈中下移动帧
<code>d(own) [count]</code>
列出源码 (当前行周围 11 行 默认延续上次)
<code>l(list) [first[, last]]</code>

列出当前函数全部源码
<code>ll longlist</code>
显示当前函数参数
<code>a(rgs)</code>
打印表达式值
<code>p expression</code>
美观打印表达式
<code>pp expression</code>
查看当前帧的局部变量字典
<code>p locals()</code>
查看当前帧的全局变量字典
<code>p globals()</code>

查看对象的所有实例变量 (返回__dict__)
<code>p vars(obj)</code>
列出对象的所有属性和方法 (含继承)
<code>p dir(obj)</code>
显示表达式类型
<code>whatis expression</code>
例: <class 'int'> / <class 'list'>
<code>whatis x / [1,2]</code>
显示源代码 (函数/类)。无法获取源码 (如内置函数) 会报错
<code>source expression</code>
每次停止时显示表达式 (变化时提示)
<code>display [expression]</code>
停止显示表达式
<code>undisplay [expression]</code>
显示当前函数返回值。需先执行 r(eturn) 到函数结束, 否则报错
<code>retval</code>

其他功能

执行任意 Python 语句
<code>! statement</code>
显示帮助 (help pdb 显示完整文档)
<code>h(elp) [command]</code>
退出调试器 (程序中止)
<code>q(uit)</code>

注意事项

PDB 支持 Tab 补全 快捷变量 .pdbrc 文件配置别名
检查当前模块
<code>p \$_name__</code>
检查当前文件位置
<code>p \$_file__</code>
查看函数文档
<code>p \$_doc__</code>
查看异常
<code>p \$_exception__</code>
查看堆栈
<code>p \$_traceback__</code>

实用技巧

进入交互式解释器 (带当前帧上下文), 适合临时多步探索
<code>interact</code>
小习惯: 先用 where/w 看堆栈, 再用 u/d 切到目标帧, 然后用 args/locals()/p 观察变量
一行打印变量名和值 (Python 3.8+)
<code>print(f"{x=}")</code>
支持表达式的自动打印
<code>print(f"{x+y=}")</code>
美化打印复杂数据结构 (自动换行缩进)
<code>from pprint import pprint; pprint(da ta)</code>

pip 包管理

安装最新版本的包
<code>pip install package_name</code>
批量安装依赖包
<code>pip install -r requirements.txt</code>
升级到最新版本 (简写 -U)
<code>pip install --upgrade package_name</code>
卸载包 (-y 跳过确认)
<code>pip uninstall package_name</code>
导出当前环境所有包版本
<code>pip freeze > requirements.txt</code>
查看包的详细信息, 包括安装位置、版本、依赖等
<code>pip show package_name</code>
列出所有当前环境中的包及其版本
<code>pip list</code>

设置全局镜像源 (加速下载)
<code>pip config set global.index-url http s://pypi.tuna.tsinghua.edu.cn/simple</code>

venv 环境管理

创建名为 venv 的虚拟环境 (标准库)
第一个venv是内置模块的名字 第二个venv是环境名
<code>python -m venv <u>venv</u> (约定俗称的虚拟环境名)</code>
激活环境 (Linux/macOS)
<code>source venv/bin/activate</code>
退出当前虚拟环境
<code>deactivate</code>

删除环境 (直接删除文件夹即可)
<code>rm -rf venv</code>
Pipenv 环境管理
为当前用户安装 Pipenv (跨平台 命令相同)
<code>pip install --user pipenv</code>
在项目目录初始化 Pipenv 创建虚拟环境 同时生成 Pipfile / Pipfile.lock
<code>cd my_project && pipenv install</code>

在当前项目环境中安装依赖 自动写入 Pipfile 并更新 Pipfile.lock
<code>pipenv install requests</code>
安装开发依赖 (dev 分组)
<code>pipenv install --dev pytest</code>
在 Pipenv 虚拟环境中运行脚本/命令 (不手动激活)
<code>pipenv run python main.py</code>
进入当前项目虚拟环境交互式 Shell
<code>pipenv shell</code>
退出 Pipenv shell (环境自动退出)
<code>exit / Ctrl-D</code>
根据 Pipfile.lock 精确重建环境 (部署/CI 推荐)
<code>pipenv sync</code>
查看依赖树 (包含间接依赖)
<code>pipenv graph</code>

Conda 环境管理

创建指定Python版本的虚拟环境
<code>conda create -n env_name python=3.10</code>
激活虚拟环境
<code>conda activate env_name</code>
列出所有虚拟环境
<code>conda env list</code>
在当前环境安装包 (-c conda-forge 指定源)
<code>conda install package_name</code>
删除整个虚拟环境
<code>conda remove -n env_name --all</code>
列出所有环境中的包及其版本
<code>conda list</code>
导出跨平台环境配置文件
<code>conda env export > environment.yml</code>

根据配置文件重建环境
<code>conda env create -f environment.yml</code>
清理未使用的包和缓存 (释放空间)
<code>conda clean -a</code>

uv 极速管理

初始化新项目并指定 Python 版本 (类似 npm init)
<code>uv init project_name --python 3.11</code>
添加依赖并更新锁文件 (自动管理 venv)
<code>uv add package_name</code>
运行脚本或命令 (自动处理依赖环境)
<code>uv run script.py</code>
同步项目依赖到环境 (自动创建/更新虚拟环境)
<code>uv sync</code>
安装全局局命令行工具 (类似 pipx)
<code>uv tool install ruff</code>
临时运行工具 (无需安装, 别名 uv tool run)
<code>uvx pycowsay "hello"</code>
安装 Python 解释器版本
<code>uv python install 3.12</code>
调整 (锁定) 当前项目使用的 Python 版本
<code>uv python pin 3.11</code>
兼容 pip 的安装命令 (极速模式)
<code>uv pip install -r requirements.txt</code>

Python 命名规范

-	类型	公有 (Public)	内部 (Internal)
包		lower_with_under	-
模块		lower_with_under	_前缀
类		CapWords	_前缀
异常		CapWords	-
函数		lower_with_under()	_前缀
常量		CAPS_WITH_UNDER	_前缀
全局/类变量		lower_with_under	_前缀
实例变量		lower_with_under	_前缀
方法		lower_with_under()	_前缀
参数		lower_with_under	-
局部变量		lower_with_under	-